DataTrace: Production Architecture Strategy

A Comprehensive Technical Guide for Engineering Portfolios and Technical Interviews

Executive Strategy Summary

This technical guide details the blueprint required to upgrade the **DataTrace** codebase from a sequential ingestion framework into an enterprise-grade **Data Observability & Intelligent Auditing Platform**. By implementing these architectural patterns, the project transitions into a high-impact portfolio item demonstrating mastery over asynchronous state-machines, AI systems engineering (RAG), UI design systems, and modern DevOps practices.

1. Current State vs. Target Production State

Understanding the gaps between the current codebase and an enterprise-scale architecture is critical for articulately presenting your system design in senior-level software engineering interviews.

Architectural Layer	Current Implementation	Target Production Upgrade
Execution Pattern	Synchronous Flask processing. Server blocks execution during data sanitization and trust-score compilation.	Asynchronous Distributed Job Queue via Celery and Redis. Immediate HTTP 202 acceptance with polling/ SSE hooks.
Pipeline Auditing	Transient, file-buffered execution arrays tracking processing statistics and trust vectors inside SQLite/ memory.	RAG-Driven Intelligent Agent utilizing semantic embedding lookups on structural logs mapped inside a vector database.
User Interface	Static dashboard reporting high-level scores and static tabular views of metrics components.	Interactive, Node-Based Data Lineage graph rendering real-time nodes and live WebSocket processing updates.
Infrastructure	Local manual execution scripts requiring manual environment management.	Fully containerized multi-container setup managed via Docker Compose, controlled with a GitHub Actions CI pipeline.

2. Asynchronous Queue Architecture (Celery + Redis)

To avoid HTTP connection timeouts when ingesting massive ecommerce analytics files (such as the Shein tracking datasets inside your tests directories), data transformations must execute detached from the core request thread. This is handled by integrating **Celery** with a **Redis** backend layer.

Step-by-Step Backend Refactoring

1. Introduce a separate task layer `Backend/tasks.py`. This decouples the processing engine logic from the route listeners.

```
from celery import Celery
import time
from services.processing import clean_dataset # Mapping your legacy engine
from services.trust_score import calculate_trust # Scoring engine

# Initialize Celery app context
celery_app = Celery('datatrace_tasks',
                    broker='redis://localhost:6379/0',
                    backend='redis://localhost:6379/0')

@celery_app.task(bind=True)
def execute_pipeline_task(self, file_path):
    self.update_state(state='PROGRESS', meta={'stage': 'Ingestion & Schema Check'})
    time.sleep(1) # Structural load delay

    # Trigger your core cleaning mechanisms
    self.update_state(state='PROGRESS', meta={'stage': 'Data Cleaning & Normalization'})
    cleaned_df = clean_dataset(file_path)

    self.update_state(state='PROGRESS', meta={'stage': 'Trust Score Verification'})
    metrics = calculate_trust(cleaned_df)

    return {'status': 'Success', 'metrics': metrics, 'file': file_path}
```

2. Refactor `Backend/app.py` to offload work instantly and monitor tasks cleanly:

```
from flask import Flask, request, jsonify
from tasks import execute_pipeline_task

app = Flask(__name__)

@app.route('/api/v1/pipeline/ingest', methods=['POST'])
def handle_ingestion():
    if 'file' not in request.files:
        return jsonify({"error": "Missing multipart payload"}), 400

    file = request.files['file']
    target_path = f"/tmp/{file.filename}"
    file.save(target_path)

    # Trigger the task and hand off execution thread immediately
    task = execute_pipeline_task.delay(target_path)
    return jsonify({"task_id": task.id, "status_url": f"/api/v1/pipeline/status/{task.id}"}),

@app.route('/api/v1/pipeline/status/<task_id>', methods=['GET'])
def get_task_status(task_id):
    task = execute_pipeline_task.AsyncResult(task_id)
    return jsonify({
        "task_id": task_id,
        "state": task.state,
        "meta": task.info if task.state == 'PROGRESS' else str(task.info)
    }), 200
```

3. RAG-Driven Intelligent Auditing Engine

A typical data engineer wastes substantial time checking execution records to trace data drop-offs. Implementing a RAG (Retrieval-Augmented Generation) loop over your pipeline execution metadata makes DataTrace exceptionally innovative for your portfolio.

The Auditing Implementation Logic

During a data-cleaning execution run, every action is logged into a fine-grained structural message list. For instance: `"Row 442: Dropped missing email property."` or `"Row 1024: Merged with row 88 using Levenshtein match score of 0.94"`.

These individual string entries are immediately embedded into vector vectors via an embedding layer and sent to a vector database.

Technical Implementation Snippet (`Backend/audit_agent.py`)

```
import os
from langchain_openai import OpenAIEmbeddings, ChatOpenAI
from langchain_community.vectorstores import SupabaseVectorStore
from langchain.chains import RetrievalQA
from supabase.client import create_client, Client

def store_execution_metadata(task_id: str, logs: list[str]):
    supabase_url = os.getenv("SUPABASE_URL")
    supabase_key = os.getenv("SUPABASE_KEY")
    supabase: Client = create_client(supabase_url, supabase_key)

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    # Insert structured data logs with task_id scoping metadata filters
    metadatas = [{"task_id": task_id, "raw_log": log} for log in logs]
    SupabaseVectorStore.from_texts(
        texts=logs,
        embedding=embeddings,
        client=supabase,
        table_name="pipeline_logs",
        query_name="match_logs",
        metadatas=metadatas
    )

def query_pipeline_audit(task_id: str, user_query: str) -> str:
    supabase_url = os.getenv("SUPABASE_URL")
    supabase_key = os.getenv("SUPABASE_KEY")
    supabase: Client = create_client(supabase_url, supabase_key)

    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")

    vectorstore = SupabaseVectorStore(
        client=supabase,
        embedding=embeddings,
        table_name="pipeline_logs",
        query_name="match_logs"
    )

    # Strict metadata boundary checks to isolate logs belonging to other pipeline sessions
    retriever = vectorstore.as_retriever(
        search_kwargs={'filter': {'metadata->>task_id': task_id}}
    )

    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    qa = RetrievalQA.from_chain_type(llm=llm, chain_type="stuff", retriever=retriever)

    return qa.run(user_query)
```

4. Advanced React Lineage Interface & Design Enhancements

Hiring teams judge systems by their structural interfaces. Looking at your current React configuration layout, replacing a tabular presentation grid with a node-graph engine dramatically elevates your engineering profile.

Integrating React Flow for Data Lineage

Using `reactflow`, you should map each modular service inside your backend processing directory (`processing.py`, `evaluation.py`, `trust_score.py`) to a visible data pipeline node block.

```
import React from 'react';
import ReactFlow, { Background, Controls } from 'reactflow';
import 'reactflow/dist/style.css';

const initialNodes = [
  { id: '1', type: 'input', data: { label: 'Raw CSV Ingestion' }, position: { x: 50, y: 150 }
  { id: '2', data: { label: 'Whitespace & Character Stripper' }, position: { x: 250, y: 150 }
  { id: '3', data: { label: 'Fuzzy Record Deduplication' }, position: { x: 450, y: 150 } },
  { id: '4', type: 'output', data: { label: 'Trust Matrix Verification' }, position: { x: 650
];

const initialEdges = [
  { id: 'e1-2', source: '1', target: '2', animated: true },
  { id: 'e2-3', source: '2', target: '3', animated: true },
  { id: 'e3-4', source: '3', target: '4', animated: true },
];

export default function PipelineVisualizer() {
  return (
    <div style={{ width: '100%', height: '400px' }} className="backdrop-blur-md bg-white/10 r
      <ReactFlow initialNodes={initialNodes} initialEdges={initialEdges} fitView>
        <Background color="#aaa" gap={16} />
        <Controls />
      </ReactFlow>
    </div>
  );
}
```

Refining Design Systems for Enterprise Appeal

- **Apply Glassmorphic Foundations:** Avoid solid gray containers. Use composite Tailwind overlays such as `backdrop-blur-lg bg-slate-900/70 border border-slate-700/50` to achieve an enterprise data platform aesthetic.
- **Real-time Terminal Outputs:** Add a terminal component to the dashboard displaying real-time task progress streams fetched from the Celery Redis backend via Server-Sent Events (SSE).

5. Containerization & Production CI/CD Setup

A professional engineer does not deploy applications manually. Enclosing your system inside a **Docker** microservice architecture proves that your code can scale reliably across cloud environments.

Production Orchestration Script (`docker-compose.yml`)

```
version: '3.8'

services:
  web_api:
    build:
      context: ./Backend
      dockerfile: Dockerfile
    ports:
      - "5001:5001"
    environment:
      - REDIS_URL=redis://redis_broker:6379/0
      - OPENAI_API_KEY=${OPENAI_API_KEY}
    depends_on:
      - redis_broker

  celery_worker:
    build:
      context: ./Backend
      dockerfile: Dockerfile
    command: celery -A tasks.celery_app worker --loglevel=info
    environment:
      - REDIS_URL=redis://redis_broker:6379/0
      - OPENAI_API_KEY=${OPENAI_API_KEY}
    depends_on:
      - redis_broker

  redis_broker:
    image: redis:7-alpine
    ports:
      - "6379:6379"
```

Automated Infrastructure Validation (`.github/workflows/ci.yml`)

This GitHub Actions profile runs on every push code event, ensuring your data manipulation components remain reliable.

```yaml
name: DataTrace Core Integration Engine

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]

jobs:
  test_suite:
    runs-on: ubuntu-latest
    steps:
    - name: Clone Code Repository
      uses: actions/checkout@v4

    - name: Initialize Runtime Environment
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
        cache: 'pip'

    - name: Inject Project Requirements
      run: |
        python -m pip install --upgrade pip
        pip install -r Backend/requirements.txt
        pip install pytest flake8

    - name: Enforce Code Style Standards
      run: |
        # Fail the pipeline run if syntax issues are discovered
        flake8 Backend --count --select=E9,F63,F7,F82 --show-source --statistics
        flake8 Backend --count --exit-zero --max-complexity=10 --max-line-length=127 --statis

    - name: Execute Pipeline Assertions
name: Run Pytest Assertions
      run: |
        pytest Backend/tests/
```

6. Strategic Resume Positioning Examples

To maximize resume screening callback loops, rewrite project descriptions to focus explicitly on architectural design choices, data scale, and system business impacts rather than simple coding actions:

⭐ Recommended Professional Resume Pattern

DataTrace – Distributed Data Lineage & Intelligent Auditing Platform

- **Asynchronous System Design:** Re-architected a legacy ingestion processing framework into a decoupled backend pipeline using **Celery and Redis task queues**, improving memory bounds and eliminating HTTP response bottlenecks for multi-gigabyte data feeds.
- **AI Auditing Architecture:** Designed a conversational context auditing engine using **LangChain, GPT-4, and Supabase Vector storage**, indexing transactional execution steps to handle natural-language lineage discovery and reduce debug cycles.
- **Data Visualization Flow:** Developed a high-performance analytics interface using **Next.js, TypeScript, and React Flow**, mapping pipeline execution node steps visually to simplify pipeline performance troubleshooting.
- **Enterprise GitOps Pipelines:** Implemented **Docker container orchestration** structures integrated with **GitHub Actions CI** workflows to enforce automatic environment syntax checks and programmatic unit tests.